第 18 章練習解答　Linux 系統

> 題號與本章練習題 PDF 一一對應。答案保留成立條件、關鍵推導與必要的反例。

OS18-E01　fork 與 exec

1. 一般fork的task/mm/page tables已各自建立；private user pages先共用唯讀frame，child只讀則不需複製它們。若child立即exec，舊user mappings被替換，避免複製即將丟棄的頁；metadata成本仍存在。
2. 本題single-thread child的PID保持，exec替換其user code/data/stack/entry；parent仍是獨立task。多thread exec會涉及停止其他threads等額外語意，本題未把它們當新建process。
3. 一般早期失敗會返回舊程式的呼叫處，child可記錄errno並以_exit等明確退出讓parent wait取得失敗。若已跨v3.2 ELF flush_old_exec的point of no return，部分失敗會收到致命signal，不能再假設有可還原舊image。parent亦須辨別正常exit與signal終止。

檢查。 每一步都要指定 task 與 address space。

OS18-E02　Module 引用

1. 不可，兩種仍存活的引用都可能執行module code。
2. 先禁止合法新入口或新ref，完成/取消I/O並確認DMA停止，關閉handles，再依本module的worker/timer/RCU等實際使用方式等待quiescence；最後ref與callbacks都不再使用code才free。並非每個module都固定等一次RCU。
3. 不能。簽章驗證可支持來源/完整性政策，無法證明執行中的指標或DMA生命週期已結束；Linux3.2是否強制簽章也不能假設與現代設定相同。

檢查。 將 refcount、outstanding I/O 與 quiescence 分開。

OS18-E03　VFS 與 page cache

1. 成功表示該次 read 期間可看到 cache 中的 bytes，不代表 media 上已有相同內容。
2. page 可能仍 dirty，block/device cache 也可能未寫回；若只是一般 read，更不需 durable。
3. 明確的 flush/commit 可要求相關 dirty 資料與 metadata 到達持久層，但實際語意仍需 API 與 FS 證明。

檢查。 區分 visibility、writeback 與 durability。

第 18 章練習解答　Linux 系統 （續）

OS18-E04　Credentials與regular-file DAC

1. 依序三權限：1000無cap為允/拒/拒；0有DAC_OVERRIDE為允/允/拒；0無DAC能力為拒/拒/拒；2000有DAC_READ_SEARCH為允/拒/拒。owner先看0400的owner bits；其他caller依普通DAC失敗後才套能力。regular-file execute在所有x bits為0時不能單靠DAC_OVERRIDE補上。
2. 0401已有other execute bit，S_IXUGO非零，所以CAP_DAC_OVERRIDE的execute DAC例外可通過；之後仍須通過mount noexec、ELF格式與其他policy。這是regular file的DAC一層，不是任意root操作皆成功。
3. 不是。seteuid後IDs1000/1000/0，saved0仍在，可依規則恢復euid0。永久降權須另處理saved IDs、groups、capability sets及privileged handles；只給一組UID數字不足以證明完整sandbox。

檢查。 使用實際owner類別、mode與capability分支，不以root標籤代替權限推導。

OS18-E05　Timeout、EIO與遲到的completion

1. 不可。timeout只表示期限內未觀察完成。若free後F40配給另一page，裝置遲到DMA仍會寫新owner資料；必須完成裝置的cancel/reset或其他停止證據，再解map。
2. 仍發布恰1次終端通知，error=EIO；本題沒有先前成功bytes，transferred=0。把錯誤當作不通知會讓waiter永遠等待，也無法結束引用生命週期。
3. 終端狀態的同generation事件不得再次通知/free；新request的generation4與舊3不符，不能被舊事件完成。此身分檢查仍不代替停止硬體DMA，兩項必要條件不同。
4. 不能。即使DMA=0且terminal已發布，consumer仍可能copy/讀F40。只有DMA停止、terminal完成、所有需要內容的引用釋放後，才安全重用；通知一次與buffer可free不是同義詞。

檢查。 狀態表同時保留DMA所有權、通知次數、bytes與consumer引用。

第 18 章練習解答　Linux 系統 （續）

OS18-E06　Clone sharing與ELF零尾部

1. 計parent+child，普通fork為2 tasks、2 mm、2 fd tables，但兩表原fd4仍引用同一O1 description。thread-like含VM/SIGHAND/THREAD/FILES為2 tasks、1 mm、1 fd table、同一O1，且TGID共享。兩種模式都不共享task_struct本體。
2. 皆拒絕EINVAL。THREAD需要SIGHAND；SIGHAND需要VM。不能把這些flags當任意獨立布林值，也不能只因VM共享就推論TGID一定共享。
3. 頁0x400000含4096file bytes；頁0x401000含2048file+2048zero；頁0x402000含4096zero。合計6144file+6144zero=12288virtual bytes。VMA描述內容來源與權限，resident可延到page fault才建立。
4. 一般在破壞舊image之前的驗證失敗可回傳錯誤而保持舊程式；v3.2 flush_old_exec之後已不可假設還原，部分失敗會送SIGKILL/SIGSEGV終止。成功exec在single-thread案例保持PID但替換user image，不建立另一個普通fork child。

檢查。 把task/mm/fd/O1物件、virtual/physical駐留及exec提交邊界分開。

OS18-E07　IRQ重入與module卸載競爭

1. preemption只阻止scheduler切換task，IRQ仍可打斷持L的process。handler等待L，但被打斷的持有者不能繼續釋放L，形成自我死鎖。
2. 不夠，CPU1仍並行。須使用所有相關contexts遵守的atomic spinlock，並按本地IRQ共享關係irq-save、離場restore原狀態。NMI或未用同鎖者仍不在此保證內。
3. refs依序5、5、2、0、0。GOING先禁止新入者；refs0時仍須等待該module的worker/timer/RCU或DMA等相應protocol全部靜止，最後才free。沒有「所有module固定一次RCU」的通則。
4. 另一CPU可在檢查0之後、關入口之前新增ref並開始呼叫，之後free會執行已釋放code。必須把新引用的取得與state檢查納入同一安全協定，再排空既有工作。

檢查。 分別證明本地重入、跨CPU互斥與最後引用競爭，不混成一個disable命令。

第 18 章練習解答　Linux 系統 （續）

OS18-E08　TCP分段與延遲成因

1. MSS=1500−20−20=1460。6000=4×1460+160，五個payload；headers5×40=200，L3 bytes6200。若計ACK/Ethernet或offload，分母另變，不能挪用本數字。
2. 不能，只表示API接受6000bytes。TCP ACK也只到達對端協定層的對應確認，不代表remote app已消費，更不代表遠端filesystem durable。
3. enqueue→dispatch=2ms，dispatch→transmit完成=1ms，其後→ACK=10ms；以enqueue到ACK為終點共13ms。不是把每層可能重疊的聚合平均任意相加。
4. 不能。聚合計數未連到同一flow及時間順序。需同request/flow的timestamps、queue entry/exit、drop與ACK/retrans evidence，或控制其他因素的干預比較，才有依據分辨queue延遲與丟失/重排等機制。

檢查。 API bytes、wire bytes、ACK與application完成分母各自明列。

OS18-E09　Pipe EOF、signal計數與原子semop

1. 不一定。parent還保留writer，所以buffer空時consumer仍等待或回EAGAIN，非EOF。所有writer引用都關閉且buffer已空，read才回0；fork複製fd tables不會自動關不用端點。
2. 不必，同standard signal可合併成一個pending事件，不能作精確計數器。real-time signals可排隊並以相應介面攜帶受限payload，但仍有queue上限與處理限制。
3. 不應。批次要求原子性，不能先永久扣S0再等待S1；在整組可完成前不公布部分效果。若選nonblocking則應按介面回錯，亦不部分扣除。
4. consumer見false後，producer可能先設true+wake，此時consumer尚未入queue；之後才sleep便漏掉wake。須先建立可被喚醒的等待狀態，再重查condition；true則不sleep，醒來也重查。實際kernel macro還提供所需鎖定與ordering，不能只靠口頭先後。

檢查。 stream資料、writer生命週期、pending signal與semaphore批次是不同狀態。

第 18 章練習解答　Linux 系統 （續）

原創驗算程式附錄: fork_cow_model.py

```
"""Linux-style private-page fork model, not a kernel implementation.
Mapping counts exclude other kernel references. No huge pages or MAP_SHARED.
"""
from dataclasses import dataclass
from collections import Counter
import json

@dataclass
class PTE:
    frame:int
    writable:bool
    cow:bool=False

@dataclass
class MM:
    name:str
    table:dict[int,PTE]

@dataclass
class Task:
    pid:int
    mm:MM
    state:str='ready'

class Model:
    def __init__(self):
        self.frames={17:bytearray([41])+bytearray(4095)}
        self.parent=Task(100,MM('M100',{0x4000:PTE(17,True)}))
        self.tasks=[self.parent];self.faults=0;self.copied_bytes=0
    def fork(self,clone_vm=False):
        if clone_vm: mm=self.parent.mm
        else:
            table={}
            for va,pte in self.parent.mm.table.items():
                pte.writable=False;pte.cow=True
                table[va]=PTE(pte.frame,False,True)
            mm=MM('M101',table)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 18 章練習解答　Linux 系統 （續）

```
        child=Task(101,mm);self.tasks.append(child);return child
    def mappings(self):
        distinct={id(t.mm):t.mm for t in self.tasks}
        return dict(Counter(p.frame for mm in distinct.values() for p in mm.table.values()))
    def write(self,task,value):
        pte=task.mm.table[0x4000]
        if not pte.writable:
            assert pte.cow
            self.faults+=1
            if self.mappings()[pte.frame]>1:
                self.frames[23]=bytearray(self.frames[pte.frame])
                self.copied_bytes+=4096;pte.frame=23
            pte.writable=True;pte.cow=False
        self.frames[pte.frame][0]=value
    def snapshot(self,label):
        return {'stage':label,'tasks':
[{'pid':t.pid,'mm':t.mm.name,'frame':t.mm.table[0x4000].frame,'W':int(t.mm.table[0x4000].writ
able),'COW':int(t.mm.table[0x4000].cow),'x':self.frames[t.mm.table[0x4000].frame][0]} for t
in
self.tasks],'mapped_PTE_counts':self.mappings(),'faults':self.faults,'copied_data_bytes':self
.copied_bytes}

def main():
    m=Model();trace=
[m.snapshot('before')];child=m.fork();trace.append(m.snapshot('after_fork'))
    assert child is not m.parent and child.mm is not m.parent.mm
    assert child.mm.table is not m.parent.mm.table
    assert child.mm.table[0x4000] is not m.parent.mm.table[0x4000]
    child.state='blocked';assert m.parent.state=='ready'
    assert m.copied_bytes==0 and m.mappings()=={17:2}
    m.write(child,42);trace.append(m.snapshot('child_first_write'))
    assert m.frames[17][0]==41 and m.frames[23][0]==42
    assert m.mappings()=={17:1,23:1} and m.copied_bytes==4096 and m.faults==1
    shared=Model();other=shared.fork(clone_vm=True)
    assert other is not shared.parent and other.mm is shared.parent.mm
    shared.write(other,42);assert shared.frames[17][0]==42 and shared.faults==0
    wrong=Model();bad_child=wrong.parent;bad_child.pid=101
    assert wrong.parent.pid==101
    print(json.dumps({'fork_trace':trace,'CLONE_VM_parent_value':shared.frames[17]
[0],'metadata_alias_counterexample_parent_pid':wrong.parent.pid},indent=2))
if __name__=='__main__':main()
```

第 18 章練習解答　Linux 系統 （續）

原創驗算程式附錄: module_lifecycle.py

```
"""State-machine check for safe module unload.

Run:
    python module_lifecycle.py
"""

from dataclasses import dataclass


@dataclass
class Module:
    active_handles: int = 0
    queued_requests: int = 0
    loaded: bool = True
    accepting: bool = True
    callbacks: int = 0

    def can_unload(self) -> bool:
        return (self.loaded and not self.accepting and self.active_handles == 0
                and self.queued_requests == 0 and self.callbacks == 0)

    def acquire_handle(self) -> None:
        if not self.accepting:
            raise RuntimeError("new entries stopped")
        self.active_handles += 1

    def begin_unload(self) -> None:
        self.accepting = False

    def unload(self) -> None:
        if not self.can_unload():
            raise RuntimeError("module still referenced")
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 18 章練習解答　Linux 系統 （續）

```
        self.loaded = False


def main() -> None:
    module = Module(active_handles=3, queued_requests=2)
    module.begin_unload()
    try:
        module.unload()
    except RuntimeError:
        pass
    else:
        raise AssertionError("unload must fail while referenced")
    try:
        module.acquire_handle()
    except RuntimeError:
        pass
    else:
        raise AssertionError("GOING must reject a new reference")
    module.active_handles = 0
    module.callbacks = 1
    module.queued_requests = 0
    assert not module.can_unload(), "last callback still executes module code"
    module.callbacks = 0
    module.unload()
    assert not module.loaded
    print("module unloaded after quiescence")


if __name__ == "__main__":
    main()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。
https://github.com/xinhi6716/computer-science-self-study